

Leader election con processi e FIFO

Alessia Smeralda Brunello

Matricola: 544964

Hands-On 16

Indice

1	Introduzione al problema	2
2	Stato dell'arte	2
2.1	FloodMax	2
2.2	LCR	2
3	Metodologia usata	3
3.1	Rappresentazione del grafo	3
3.2	Comunicazione tramite FIFO	3
3.3	Implementazione di FloodMax	4
3.4	Implementazione di LCR	4
4	Risultati sperimentali	4
4.1	Compilazione	4
4.2	Output filtrato di FloodMax	5
4.3	Output filtrato di LCR	5
4.4	Confronto tra gli algoritmi	6
5	Conclusione e sviluppi futuri	6

1 Introduzione al problema

L'obiettivo dell'esercitazione è implementare due algoritmi di *leader election* in un sistema distribuito simulato tramite processi Unix e FIFO.

Ogni processo rappresenta un nodo della rete, mentre ogni FIFO rappresenta un canale di comunicazione orientato tra due nodi.

Il problema della *leader election* consiste nell'eleggere un unico processo leader tra più processi distribuiti. Nel caso considerato, ogni processo possiede un identificatore univoco e il leader coincide con il processo avente identificatore massimo.

La traccia richiede l'implementazione di due algoritmi:

- **FloodMax**, applicato al grafo assegnato;
- **LCR**, applicato a un grafo ciclico orientato di ordine 5.

2 Stato dell'arte

La *leader election* è un problema classico dei sistemi distribuiti. In molte applicazioni, è necessario individuare un processo coordinatore, ad esempio per gestire una risorsa condivisa, sincronizzare attività o avviare decisioni globali.

In assenza di un coordinatore centrale, i processi devono comunicare tra loro usando solo informazioni locali. Gli algoritmi di elezione permettono quindi di raggiungere un accordo su un unico leader.

In questa esercitazione sono stati considerati due approcci differenti.

2.1 FloodMax

FloodMax è un algoritmo pensato per grafi generici. Ogni processo mantiene il massimo identificatore conosciuto fino a quel momento. Inizialmente conosce solo il proprio ID; a ogni round invia tale massimo ai vicini e aggiorna il proprio stato in base ai messaggi ricevuti.

Dopo un numero sufficiente di round, pari al diametro considerato del grafo, l'identificatore massimo si è propagato a tutti i processi. Il nodo il cui ID coincide con il massimo globale si dichiara leader.

2.2 LCR

LCR è un algoritmo per reti ad anello orientato. Ogni processo invia inizialmente il proprio identificatore al vicino successivo. Quando riceve un ID, lo confronta con il proprio:

- se è minore, lo scarta;
- se è maggiore, lo inoltra;
- se è uguale al proprio, si dichiara leader.

In questo modo gli identificatori minori vengono eliminati progressivamente, mentre quello massimo completa l'intero giro dell'anello e torna al processo di origine.

3 Metodologia usata

Il progetto è stato realizzato in linguaggio C utilizzando processi e FIFO. Il processo principale crea la rete, genera un processo figlio per ogni nodo e attende la terminazione di tutti i figli.

La struttura del progetto è modulare ma compatta: il codice è suddiviso tra:

- gestione del grafo,
- comunicazione tramite FIFO
- implementazione degli algoritmi

Il `Makefile` invece automatizza la compilazione e l'esecuzione.

3.1 Rappresentazione del grafo

Il grafo usato da FloodMax viene letto dal file `graph.txt`. La prima riga indica il numero di nodi e di archi, mentre le righe successive descrivono gli archi orientati:

```
6 12
5 6
6 5
6 4
5 1
1 4
1 3
1 2
3 4
2 4
2 3
2 5
3 5
```

Per LCR, invece, il ring viene costruito automaticamente nel codice:

```
1 -> 2 -> 3 -> 4 -> 5 -> 1
```

3.2 Comunicazione tramite FIFO

Ogni arco orientato viene implementato tramite una FIFO. Ad esempio, l'arco:

```
1 -> 3
```

corrisponde a un canale in cui il processo 1 scrive e il processo 3 legge.

I messaggi scambiati tra processi sono rappresentati dalla struttura:

```
typedef struct {
    int sender;
    int round;
    MessageKind kind;
    int data;
} Message;
```

Il campo `kind` indica se il messaggio contiene un valore oppure se è nullo, mentre `data` contiene l'identificatore inviato.

3.3 Implementazione di FloodMax

In FloodMax ogni processo mantiene uno stato locale:

```
typedef struct {
    int my_id;
    int max_id;
    LeaderState leader;
    int round;
} FloodState;
```

All'inizio ogni processo conosce solo il proprio identificatore, quindi `max_id` è inizializzato a `my_id`. A ogni round il processo invia il massimo ID conosciuto a tutti i vicini uscenti e riceve i messaggi dai vicini entranti.

La regola di aggiornamento è:

```
max_id = massimo tra il valore attuale e i valori ricevuti
```

Nel grafo considerato, il massimo identificatore è 6. È stato usato `diam = 3`, cioè il numero di round necessario affinché l'identificatore massimo, pari a 6, raggiunga anche i nodi più lontani nella topologia considerata.

Dopo tre round, tutti i processi conoscono il massimo ID e possono decidere se sono leader oppure no.

3.4 Implementazione di LCR

In LCR ogni processo mantiene il seguente stato:

```
typedef struct {
    int my_id;
    int max_id;
    LeaderState leader;
    int snd_flag;
} LcrState;
```

Il campo `snd_flag` indica se il processo deve inviare un messaggio nel round corrente. Inizialmente tutti i processi inviano il proprio ID al successore.

Nel ring di ordine 5, il valore massimo è 5. Gli ID minori vengono scartati quando arrivano a un processo con identificatore più grande, mentre il valore 5 continua a essere inoltrato fino a tornare al processo 5, che si dichiara leader.

4 Risultati sperimentali

4.1 Compilazione

La compilazione del progetto avviene tramite:

```
make
```

Il Makefile fornisce anche due comandi per l'esecuzione:

- **make run-floodmax**
che contiene il comando:
`./leader_election floodmax graph.txt 3`
- **make run-lcr**
che contiene il comando:
`./leader_election lcr 5`

e due comandi per un'esecuzione filtrata:

- **make run-floodmax-sort**
che contiene il comando:
`./leader_election floodmax graph.txt 3 | grep "^\[FINE\]" | sort`
- **make run-lcr-sort**
che contiene il comando:
`./leader_election lcr 5 | grep "^\[FINE\]" | sort`

4.2 Output filtrato di FloodMax

```
alessia23@aleb23:~/LabReSid24-25/hands-on/ho16/codice$ make run-floodmax-sort
./leader_election floodmax graph.txt 3 | grep "^\[FINE\]" | sort
[FINE][FLOODMAX] P1: max_id = 6, leader = false
[FINE][FLOODMAX] P2: max_id = 6, leader = false
[FINE][FLOODMAX] P3: max_id = 6, leader = false
[FINE][FLOODMAX] P4: max_id = 6, leader = false
[FINE][FLOODMAX] P5: max_id = 6, leader = false
[FINE][FLOODMAX] P6: max_id = 6, leader = true
```

Tutti i processi arrivano a conoscere il massimo ID, cioè 6, e solo il processo 6 viene eletto leader.

4.3 Output filtrato di LCR

```
alessia23@aleb23:~/LabReSid24-25/hands-on/ho16/codice$ make run-lcr-sort
./leader_election lcr 5 | grep "^\[FINE\]" | sort
[FINE][LCR] P1: max_id = 5, leader = false
[FINE][LCR] P2: max_id = 5, leader = false
[FINE][LCR] P3: max_id = 5, leader = false
[FINE][LCR] P4: max_id = 5, leader = false
[FINE][LCR] P5: max_id = 5, leader = true
```

Il processo 5 viene eletto leader perché possiede l'identificatore massimo del ring.

4.4 Confronto tra gli algoritmi

Aspetto	FloodMax	LCR
Topologia	Grafo generico	Ring orientato
Invio dei messaggi	A tutti i vicini uscenti	Solo al successore
Informazione usata	Massimo ID conosciuto	ID candidato
Decisione del leader	Dopo <code>diam</code> round	Quando un ID torna al proprietario
Leader ottenuto	Processo 6	Processo 5

Tabella 1: Confronto tra FloodMax e LCR.

I risultati confermano il comportamento atteso: FloodMax diffonde il massimo identificatore nel grafo, mentre LCR fa circolare gli identificatori nel ring eliminando progressivamente quelli minori.

5 Conclusione e sviluppi futuri

L'esercitazione ha permesso di implementare due algoritmi di leader election usando processi Unix e FIFO.

FloodMax è stato applicato a un grafo orientato generico, mentre LCR è stato applicato a un ring orientato di ordine 5.

In entrambi i casi il sistema elegge correttamente un unico leader.

Possibili sviluppi futuri includono:

- il calcolo automatico del diametro per FloodMax,
- il supporto a identificatori arbitrari non coincidenti con il numero del nodo,
- una sincronizzazione più esplicita tra i round e il salvataggio dei log su file separati per ogni processo.